



Software Engineering Department

Braude College of Engineering

OPM Code Generator

Project code: 25-2-D-15

Capstone Project Phase B – 61998

January 2026

Sagi Yosofov

Liroy Ben Shimon

Git repository link:

<https://github.com/SagiYosofov/opm-code-generator.git>

Supervisor:

Dr. Natali Levi

Table of Contents

Abstract	3
<u>1. Introduction</u>	4
<u>2. Background and Related Work</u>	5
<u>2.1 The V-Model</u>	5
<u>2.2 xUML</u>	6
<u>2.3 MBSE</u>	6
<u>2.4 MAXIM</u>	7
<u>3. General Description</u>	7
<u>3.1 System Objectives</u>	8
<u>3.2 Target User Identification</u>	8
<u>3.3 Gemini: The Leading Choice for Learning OPM</u>	9
<u>4. Development Process</u>	9
<u>5. System Architecture</u>	11
<u>5.1 System Package Diagram</u>	12
<u>5.1.1 Frontend</u>	13
<u>5.1.2 Backend</u>	13
<u>5.1.3 Database</u>	14
<u>5.2 System Flow</u>	14
<u>6. Challenges and Obstacles</u>	16
<u>6.1 Selecting the Appropriate AI Model</u>	16
<u>6.2 Integrating with Gemini API and Managing AI Interactions</u>	17
<u>6.3 Designing an Intuitive User Interface</u>	17
<u>6.4 System Performance Optimization</u>	17
<u>6.5 Lesson Learned</u>	18
<u>7. Project Metrics</u>	19
<u>7.1 User Experience and Satisfaction</u>	19
<u>7.2 System Performance and Response Time</u>	20

<u>7.3 Future Work</u>	21
<u>7.4 Conclusion</u>	21
<u>8.References</u>	22
<u>Appendices</u>	22
<u>Appendix A – User Guide</u>	22
<u>Appendix B – Maintenance Guide</u>	36

Abstract

The OPM Code Generator project addresses the persistent gap between system specification and software implementation. Traditional development processes often suffer from communication breakdowns between specification and development stages, leading to misaligned systems requiring costly corrections. This project presents an innovative solution using artificial intelligence to automatically translate Object-Process Methodology (OPM) models into initial implementation in code. By transforming formal system models directly into code skeletons, software engineers can begin with an active code base aligned with original specifications rather than abstract documentation. This approach reduces development time, minimizes interpretation errors, and improves software quality. The project contributes to Model-Based Systems Engineering by demonstrating how AI systems can understand formal modeling languages and generate implementation code, bridging the critical gap between design and development phases.

Abbreviations

MBSE - Model-based systems engineering
OPM - Object-Process Methodology
OPL - Object-Process Language
UML - Unified Modeling Language
AI - Artificial Intelligence
xUML – Executable UML
MAXIM - Methodical Approach to Executable Integrative Modeling

1.Introduction

In the modern software development paradigm, a persistent challenge exists: the misalignment between customer requirements and the actual specification and development of systems [1]. The traditional specification process begins when a systems analyst translates customer requirements into a specification document. This document is intended to serve as the foundation for software engineers during the development phase. However, in practice, significant communication gaps exist between these stages [1].

Software engineers struggle to fully comprehend specification documents. In many cases, specification documents are either neglected or only partially addressed. The result is a system that fails to align with the original customer requirements. Therefore, numerous costly corrections are required in later development stages [1].

The cost of error correction increases exponentially as one progress through the system's lifecycle stages [1]. According to the Model Fidelity Hierarchy, there exist several levels of precision in models, ranging from verbal concepts, through conceptual models and computational models, to executable models [1]. The transition between these levels enables the detection of various errors at each stage; however, these transitions typically involve information loss and require substantial effort.

In addressing the challenge of bridging the gap between system specification and software development, several methodologies and frameworks have emerged, each offering unique approaches to mitigate this divide. Prominent among these are the V-Model, xUML, MBSE and MAXIM [2][4][6][7]. These solutions focus on enhancing traceability, improving communication, and enabling early validation throughout the development lifecycle, yet they exhibit various strengths and weaknesses that can affect their overall effectiveness in different contexts [2][4][6][7]. Within the scope of this project, we will examine the capability of advanced AI models to learn and process OPM diagrams describing the structure and operation of the system.

The objective of this project is to develop a system based on artificial intelligence technologies that will help to bridge the gap between specification and development. The system will accept as input an OPM model representing the system specification and generate from it a basic software implementation that will serve as a starting point for software engineers. This approach enables software engineers to start from an initial code base rather than an abstract specification, improves alignment between specification and implementation, and reduces development time while improving product quality.

The primary stakeholders in this project are software development professionals involved in translating specification documents into code. The project is expected to impact the entire development process, from the specification stage to final implementation, and to assist in closing the traditional gap between specification and actual development.

The innovation of this project lies in the integration of artificial intelligence technologies into the MBSE process, focusing on the critical gap between the specification stage and the development stage. Unlike existing approaches that focus on improving the specification process or separate development tools, the project offers an integrative approach that connects the stages through a unified and continuous model [1].

The project is expected to contribute to understanding how AI systems can be taught a "new language" - in this case, a formal modeling language like OPM and use this knowledge to streamline the entire development process and improve the quality of the resulting systems.

The structure of this document is divided into several chapters, each corresponding to a different stage of the project. Following the introduction, Chapter 2 reviews related work. Chapter 3-5 describe the system and its development. Chapter 6 discusses challenges, and Chapter 7 presents the project metrics.

2. Background and Related Work

The gap between system design and development represents a longstanding challenge in software engineering. Various methodologies and frameworks have been introduced to mitigate this divide, each offering distinct approaches to facilitating the transition from system specification to implementation. This literature review critically examines these frameworks, assessing their strengths, weaknesses, and overall effectiveness in bridging the specification-development gap in complex systems engineering contexts.

2.1 The V-Model

The V-Model represents one of the principal solutions for bridging the gap between specification and software development. The model depicts the system development process as a "V" where the left branch represents the decomposition and definition of requirements, while the right branch represents the integration and verification phases. The V-Model establishes a direct

relationship between requirements and verification, ensuring that every requirement is measurable and verifiable [2]. The model creates clear traceability between requirements and implementation [3] and enables early error detection through parallel verification processes [2].

However, implementation complexity requires deep understanding of each stage, particularly for large systems [2]. The model is less suited to environments with frequent requirement changes and requires significant documentation at each stage [3].

2.2 xUML

The xUML (Executable UML) model is a profile of the UML modeling language that enables the automatic generation of executable code directly from UML models. By utilizing standard UML diagrams along with an Action Language that defines system behavior, xUML allows for seamless translation from high-level design to executable code, ensuring consistency between design and implementation phases [4]. This capability allows for early validation of design while preserving high-level abstraction without implementation details, shortening development time and preventing inconsistencies between models and code [5].

Challenges include the need for extensive team training and process integration. The Action Language may not provide all capabilities of full-fledged programming languages [4], and the models still require mapping to execution code, creating a risk of discrepancies [5]. Furthermore, this solution requires dividing the system into subsystems with minimal interaction [2].

2.3 MBSE

Model-Based Systems Engineering (MBSE) represents a methodological approach to bridge the gap between specification and development stages. In contrast to traditional document-based approaches, MBSE offers a workflow where the model serves as the central and authoritative source of truth. The continuous transition between modeling stages, utilizing the same modeling language, ensures consistency and completeness of requirements and design, thereby preventing loss of critical information [1]. This integrative approach enables early error detection, significantly reducing correction costs [1], and effective communication among stakeholders through a unified modeling language [6].

MBSE has several limitations. The learning curve for MBSE tools is steep, requiring significant investment in training teams [1]. Integrating conceptual models with executable code is particularly challenging in complex systems, and there are difficulties in supporting all computational and executional aspects within a single modeling language [1].

2.4 MAXIM

MAXIM (Methodical Approach to Executable Integrative Modeling) is an integrated methodology based on OPM, enhanced with computational capabilities, developed to bridge the gap between systems engineering and software engineering [7]. The approach provides an integrated modeling environment enabling collaboration among systems engineers, software architects, and engineers from different disciplines in modeling a shared work plan [7]. Unlike traditional processes where the conceptual model stops evolving once detailed design begins, MAXIM enables continuity of a shared model throughout the entire development process, preventing loss of critical information during handoff [7]. The model enables early detection of development errors and provides both computational and visual execution capabilities [7].

While MAXIM offers an innovative modeling environment, its current implementation does not support modeling of non-functional properties such as reliability, security, and resource consumption [7]. The execution engine lacks capabilities such as reverse execution and stepwise debugging and does not yet support the complete set of OPM operational semantics, including probabilistic and event-driven behaviors [7].

3. General Description

The OPM Code Generator is a web-based application designed to bridge the gap between high-level system modeling and practical software implementation. The system addresses a fundamental challenge in software engineering: translating conceptual OPM diagrams into executable code skeletons across multiple programming languages.

3.1 System Objectives

The primary objective of the system is to automate the conversion of OPM diagrams into structured, maintainable code frameworks. Users upload OPM diagrams through an intuitive web interface, and the system leverages Gemini AI to analyze the diagram's structure, identifying objects, processes, states, and their relationships. The AI engine then generates corresponding code skeletons in the user's chosen programming language (Python, Java, C#, or C++), complete with class structures, method signatures, and appropriate comments that reflect the OPM model's logic. The system also supports iterative refinement, allowing users to request modifications to the generated code through natural language instructions.

3.2 Target User Identification

The OPM Code Generator is designed for multiple user groups within the software development ecosystem:

1. **System Architects and Designers:** Professionals who create high-level system designs using OPM and need to quickly prototype or communicate implementation structures to development teams.
2. **Development Teams:** Teams working on projects that utilize OPM for system specification, enabling faster transition from design phase to implementation phase.
3. **Software Engineering Students:** Students learning OPM methodology who need to understand the practical implementation of their theoretical models. The system serves as an educational bridge between modeling and coding.
4. **Educators and Researchers:** Academic professionals teaching software engineering, system modeling, or OPM methodology who can use the tool to demonstrate the connection between conceptual models and executable code.

The system assumes basic familiarity with OPM notation and programming concepts, making it accessible to intermediate users while providing value to advanced practitioners seeking to accelerate their development workflow.

3.3 Gemini: The Leading Choice for Learning OPM

We conducted a comprehensive comparison between several leading artificial intelligence models (ChatGPT, Gemini, and Claude) with the aim of identifying the most suitable platform for our OPM Code Generator project. After experimenting with each model, we found that ChatGPT exhibited a basic understanding of key concepts but struggled with comprehending the intricate hierarchies and relationships between objects and processes. Claude demonstrated a superior ability to handle complex explanations and maintain consistency throughout extended discussions. However, through practical testing with real OPM diagrams during the implementation phase, we discovered that Gemini possessed substantially deeper pre-existing knowledge of OPM notation, diagram structures, and the relationships between OPM elements. While Claude excelled at learning through conversation, Gemini demonstrated an inherent understanding of OPM visual syntax and semantics that proved crucial for accurate diagram interpretation. Additionally, Gemini's integration capabilities with our Python backend and its performance in processing diagram images made it the optimal choice. Therefore, we ultimately selected Gemini as the AI engine powering our OPM Code Generator.

4. Development Process

The development of the OPM Code Generator followed an iterative and systematic approach, divided into several key phases:

1. Research and Planning Phase: We began by conducting comprehensive research on existing AI models to identify the most suitable platform for OPM diagram analysis. We experimented with ChatGPT, Claude, and Gemini, evaluating their capabilities in understanding OPM notation and generating code. Through practical testing with real OPM diagrams, we selected Gemini as our AI engine due to its superior pre-existing knowledge of OPM structures and visual syntax. In parallel, we defined the system requirements, identifying our target users and their needs. We established the core functionalities: diagram upload, AI-powered analysis, multi-language code generation, and user authentication.

2. Architecture Design: We designed a modern three-tier architecture separating concerns between frontend, backend, and database layers. Key decisions included:

- Choosing React for the frontend to provide a responsive and interactive user interface.

- Selecting FastAPI for the backend due to its performance, async capabilities, and easy integration with AI services.
- Implementing MongoDB as our database for flexible document storage and scalability.

3. Development Phase: The development process was conducted incrementally, building and testing features in stages:

Backend Development:

- Implemented RESTful API endpoints using FastAPI.
- Integrated Gemini API for diagram analysis and code generation.
- Developed authentication system with JWT tokens.
- Created file upload and processing mechanisms for PDF diagrams.
- Built the code generation pipeline supporting multiple programming languages.

Frontend Development:

- Created responsive UI components using React.
- Implemented user authentication flow with protected routes.
- Developed drag-and-drop file upload interface.
- Built project history features.
- Added real-time feedback mechanisms (loading states, toast notifications, success pages).
- Designed professional styling with consistent UX patterns.

Database Integration:

- Configured MongoDB connection and schema design.
- Implemented user data storage and retrieval.
- Created project history tracking system.

4. Integration and Testing: We connected all system components, ensuring seamless communication between frontend, backend, and AI services. Testing included functional testing of each component individually, integration testing to verify end-to-end workflows, testing with various OPM diagram complexities, user interface testing for responsiveness and usability, and AI output validation across different programming languages.

5. Refinement and Optimization: Based on testing results and practical usage, we iteratively improved UI/UX enhancements for better user experience, error handling and user feedback mechanisms, code generation quality and accuracy, system performance and response times, and documentation and code comments.

Throughout the development process, we maintained close collaboration, dividing tasks between team members while conducting regular code reviews and integration sessions. We used version control (Git) to manage our codebase and coordinated our efforts to ensure consistent progress toward our project goal.

5. System Architecture

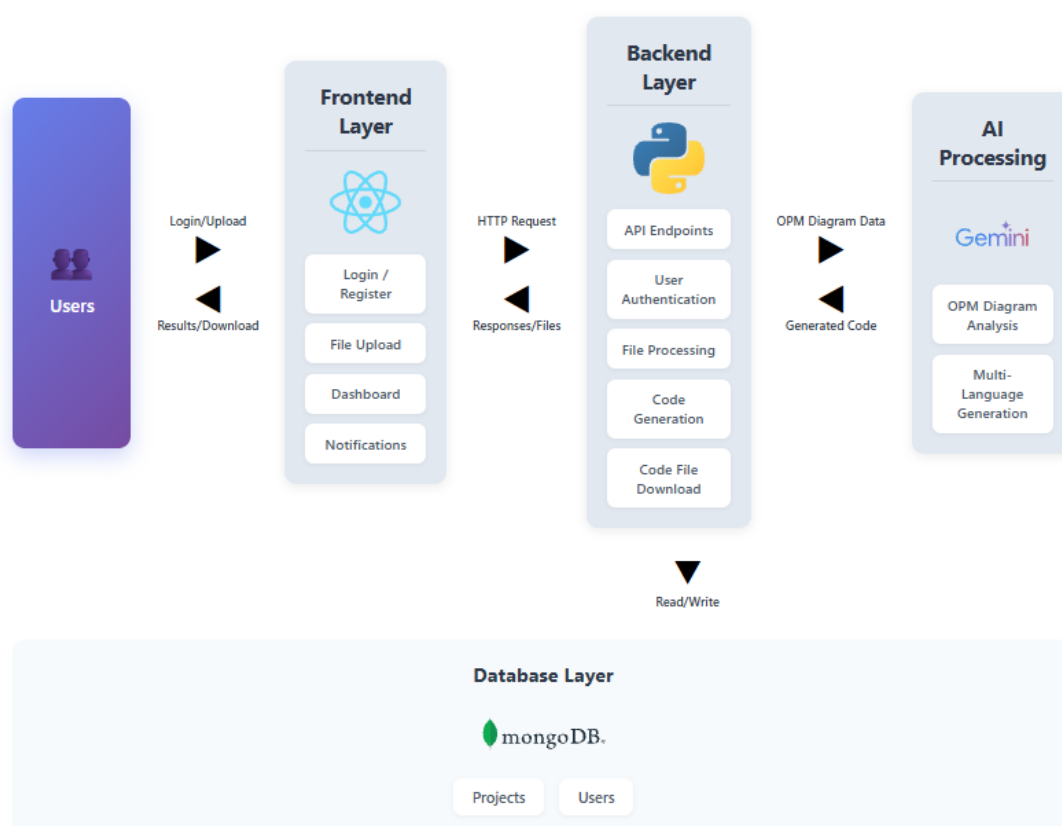


Figure 1: Architecture Diagram

The System Architecture Diagram presents the overall structure and data flow of the OPM Code Generator system. The system is built on three main layers: Frontend based on React that provides an interactive user interface for login and OPM diagram uploads, Backend developed in Python with FastAPI that handles user authentication, file processing, and code generation, and AI Processing that uses Gemini API to analyze OPM diagrams and generate code skeletons in multiple programming languages. The flow begins when users log in and upload an OPM diagram through the interface. The diagram is sent via HTTP request to the Backend, which processes the file and forwards the OPM diagram data to the

Gemini AI for analysis. The AI returns the generated code to the Backend, which then sends the results back to the Frontend for the user to download. All user information and project data are stored in a MongoDB database, with the Backend performing read/write operations as needed.

5.1 System Package Diagram

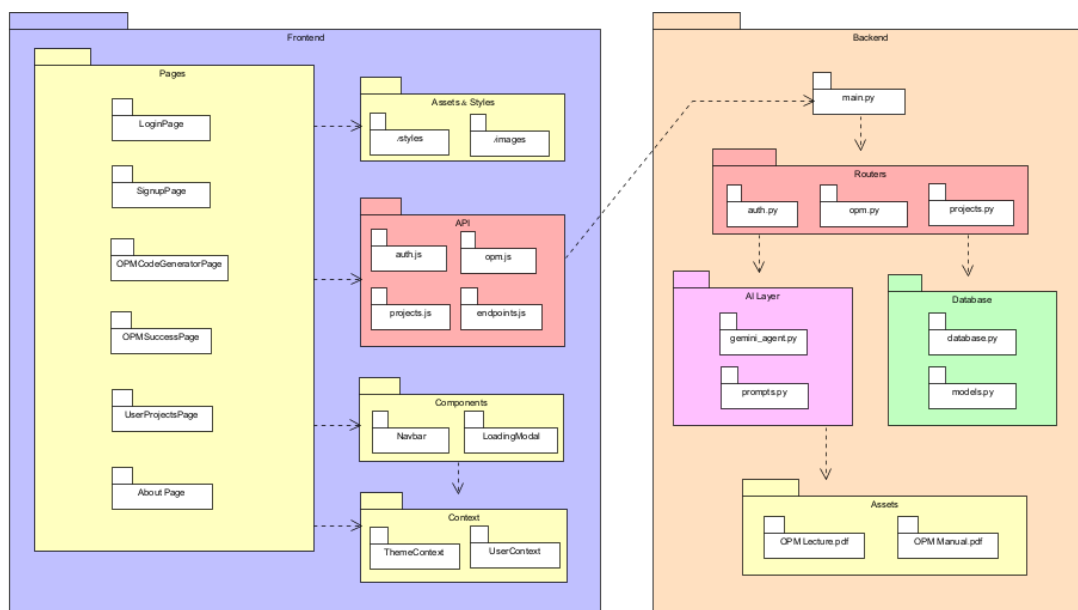


Figure 2: System Package Diagram

The Package Diagram illustrates the architectural structure of the OPM Code Generator system, which comprises two main components: Frontend and Backend. The Frontend, built with React, consists of five core sections - Pages (application views), Components (shared UI elements), Context (global state management), API (communication layer), and Assets & Styles (static resources). The Backend, built with FastAPI, comprises four main sections: Routers (HTTP request routing), AI Layer (Gemini API integration), Database (MongoDB data management), and Assets (OPM documentation files). The diagram demonstrates the data flow within the system: application Pages communicate with shared Components and Context for state management and send requests to the Backend through the API layer. On the server side, main.py routes requests to the appropriate Routers, which interact with the Database layer for data storage and management, and with the AI Layer for OPM diagram analysis and code generation. The diagram employs a pastel color scheme to distinguish functional roles: blue for the user interface, orange for the server, pink for communication, purple for artificial intelligence, green for the database, and yellow for static resources.

5.1.1 Frontend

The frontend of the OPM Code Generator is built using React, a modern JavaScript library that enables the creation of dynamic and responsive user interfaces. The application follows component-based architecture, where each UI element is encapsulated as a reusable component, promoting code maintainability and scalability. We implemented React Router for seamless navigation between different pages including login, registration, dashboard, file upload, and project history. The user interface features a professional and intuitive design with drag-and-drop file upload functionality, real-time feedback mechanisms such as loading spinners and toast notifications, and responsive layouts that adapt to different screen sizes. State management is handled through React hooks (useState, useEffect) to manage component state and side effects, while authentication state is maintained across the application using protected routes that verify user tokens before granting access to sensitive pages. The frontend communicates with the backend through RESTful API calls using the Fetch API, sending user credentials, uploaded files, and code generation requests, and receiving responses that are then rendered to the user in an accessible and visually appealing manner.

5.1.2 Backend

The backend is implemented using FastAPI, a modern Python web framework known for its high performance, automatic API documentation, and native support for asynchronous operations. The backend serves as the central orchestrator of the system, handling all business logic and coordinating between different components. It exposes RESTful API endpoints for user authentication (registration, login, token validation), file upload and processing, AI integration for diagram analysis and code generation, and project history management. The authentication system implements JWT (JSON Web Tokens) for secure, stateless user sessions, with password hashing using bcrypt to ensure sensitive data protection. File handling is managed through multipart data processing, where uploaded PDF diagrams are temporarily stored, validated, and then sent to the Gemini AI API for analysis. The backend processes the AI response, extracts relevant OPM elements including objects, processes, states, and relationships, and then executes code generation logic based on the user's selected programming language. The generated code is formatted, commented, and stored in the database along with metadata about the project. Error handling is implemented throughout the backend to catch and gracefully manage exceptions, providing meaningful error messages to the frontend for user feedback.

5.1.3 Database

MongoDB serves as the database layer for the OPM Code Generator, chosen for its flexibility, scalability, and natural fit with JavaScript and Python ecosystems. As a NoSQL document-oriented database, MongoDB allows us to store data in JSON-like documents, which aligns well with the hierarchical and varied nature of our data structures. The database scheme includes collections for users, projects, and generated code. The user's collection stores authentication information including usernames, hashed passwords and email addresses. The projects collection maintains records of each code generation request, storing the original uploaded diagram reference, the analyzed OPM structure returned by the AI, the generated code for each programming language, user preferences and refinement requests, and timestamps for creation and modification. This design enables efficient querying of user history and supports the iterative refinement process where users can request modifications to previously generated code. MongoDB's indexing capabilities are utilized to optimize query performance, particularly for user lookups during authentication and project retrieval for the history dashboard. The database connection is managed through the PyMongo driver for Python, providing a robust interface for CRUD operations (Create, Read, Update, Delete) and ensuring data consistency and reliability throughout the application lifecycle.

5.2 System Flow

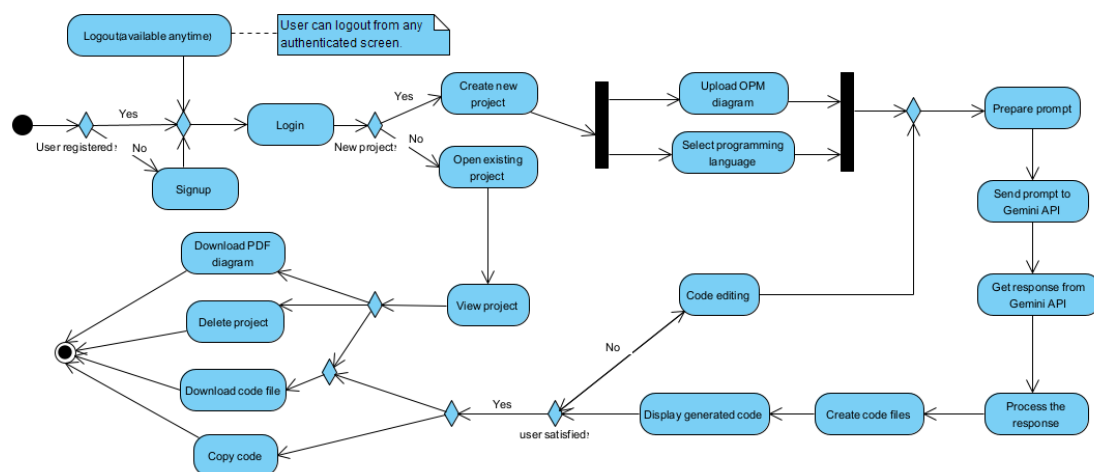


Figure 3: System Flow Diagram

The OPM Code Generator follows a well-defined workflow that guides users through the entire process from initial access to final code generation. The system's flow diagram illustrates the sequential steps and decision points that structure the user experience, with logout functionality available at any point

during authenticated sessions as indicated in the system notes. The user journey begins with an authentication checkpoint, where the system verifies whether the user is registered. New users are directed to the signup process to create an account, while existing users proceed to the login page. Upon successful authentication, users are presented with two primary options: creating a new project or opening an existing project from their history, enabling both first-time and returning users to continue their work seamlessly. For users selecting to open an existing project, the system displays the project in view mode, where they can review previously generated code and access several management options. Users can download the PDF diagram for reference, delete the project if it is no longer needed, or download the generated code file for use in their development environment. For new projects, users upload their OPM diagram in PDF format. The system provides an intuitive file upload interface with drag-and-drop functionality, making the submission process simple and straightforward. After uploading, users select their desired programming language from the supported options (Python, Java, C#, or C++), which determines the specific code generation template and syntax rules to be applied. Once the diagram and language are specified, the system prepares a structured prompt that encapsulates the diagram analysis request along with language-specific code generation instructions. This prompt is sent to the Gemini API, initiating the AI-powered analysis phase. During processing, users receive real-time visual feedback through animated loading indicators, keeping them informed of the system's status and maintaining engagement throughout the waiting period. The Gemini API analyzes the OPM diagram, identifying objects, processes, states, and their relationships, and returns a comprehensive response containing both the structural analysis and the generated code skeleton. The backend processes this response, validates its completeness, and formats it appropriately for display. The system then creates code files in the requested programming language. At this stage, the generated code is displayed to users, who are presented with a satisfaction decision point. If satisfied with the generated code, users can either download the code file to begin working with it in their development environment or copy the code directly to their clipboard for immediate use. If users are not satisfied with the generated code, they can access the code editing feature, which allows them to provide natural language instructions for refining or adjusting the code, such as adding specific functionality, modifying naming conventions, or restructuring components. When users choose to edit the code, the system loops back through the AI interaction process, preparing a new prompt that incorporates both the original diagram information and the user's refinement instructions. This prompt is sent to the Gemini API, the response is processed, and new code files are created and displayed. Users can then evaluate the revised code and again choose to download, copy, or request further modifications. This iterative refinement

capability, available only for newly created projects, ensures that users can progressively improve the generated code through multiple iterations until it meets their specific requirements and preferences. Throughout the entire workflow, the system maintains state information, saving project details and generated code to the database, allowing users to return to their projects at any time through the existing project view functionality. This comprehensive workflow design balances simplicity for straightforward use cases with flexibility for complex scenarios requiring iterative refinement, while clearly distinguishing between active project development and historical project review.

6.Challenges and Obstacles

Throughout the development of the OPM Code Generator, we encountered numerous analytical and technical challenges that required careful consideration and innovative problem-solving approaches. This section reflects on the major obstacles we faced, the solutions we implemented, the rationale behind our decisions, and the lessons we learned from the entire development process.

6.1 Selecting the Appropriate AI Model

One of the earliest and most critical challenges in the project was selecting an AI model to drive the analysis of OPM diagrams and the code generation process. At the beginning, three leading models were evaluated: ChatGPT, Claude, and Gemini. ChatGPT demonstrated a basic level of understanding but struggled with the hierarchical complexity of OPM structures, while Claude showed impressive learning capabilities and a strong grasp of theoretical concepts through dialogue, which led to its initial selection during the learning phase of the project. However, as the project transitioned to the practical implementation stage and testing was conducted on real OPM diagrams, it became evident that despite its strengths in conversational learning, Claude lacked sufficient pre-existing knowledge of OPM notation, visual syntax, and semantic rules, which negatively affected the accuracy of diagram interpretation. This insight prompted a re-evaluation of Gemini, which, through systematic testing, demonstrated a deeper inherent understanding of OPM diagram structures, the relationships between objects and processes, and the visual semantics of the modeling language. Based on these empirical findings, a strategic decision was made to adopt Gemini as the primary AI engine, as its built-in knowledge enabled immediate and accurate diagram interpretation without requiring repeated learning, offered straightforward integration with Python, and resulted in

significantly improved accuracy and reliability in code generation, despite the need for architectural adjustments and re-implementation of the AI integration layer.

6.2 Integrating with the Gemini API and Managing AI Interactions

Working with the Gemini API presented several design and maintenance challenges, including authentication management, rate limiting, handling potential API failures or timeouts, and parsing unexpected AI responses. To address these challenges, we implemented a robust API integration layer with comprehensive error handling and retry logic, including exponential backoff, and ensured that AI outputs were structured as JSON and validated before code generation. Additionally, users received real-time feedback through loading indicators and status updates, and the system architecture was designed to separate concerns clearly, allowing independent optimization of components. In retrospect, implementing caching for similar diagrams could have reduced redundant API calls, improving both performance and cost efficiency.

6.3 Designing an Intuitive User Interface

Creating a user interface that is both visually appealing and highly functional requires balancing aesthetic design with practical usability, while guiding users smoothly through the workflow and providing clear feedback on actions and errors. To achieve this, we adopted React for building reusable and maintainable components and invested in UX design with features such as drag-and-drop file upload, real-time loading indicators with animations, toast notifications for success and error messages, and a clean, professional aesthetic with gradient backgrounds and consistent styling. Informal usability testing with peers provided feedback that informed iterative improvements, and the high SUS score of 83.5 validated the effectiveness of these design decisions. In retrospect, conducting formal user research earlier and using wireframes or prototypes could have further supported concept validation and enhanced the overall user experience.

6.4 System Performance Optimization

Maintaining high system performance alongside rich features was an ongoing challenge, such as AI processing, file uploads, database queries, and frontend

rendering all required optimization. To address this, we leveraged FastAPI's asynchronous capabilities to handle concurrent requests efficiently, implemented efficient file handling mechanisms on the backend, and used React optimization features, such as lazy loading and efficient state management, on the frontend to minimize unnecessary re-renders. Additionally, the system architecture was designed with clear separation of concerns, allowing individual components to be optimized independently. While response times of 30–60 seconds were acceptable for AI-powered generation, further optimization opportunities exist, including caching, prompt engineering improvements, and potential parallel processing for multi-language code generation.

6.5 Lessons Learned

The OPM Code Generator project successfully achieved its primary objectives, resulting in the development of a functional system that automates the translation of OPM diagrams into executable code skeletons accurately and efficiently. A SUS score of 83.5 indicates high user satisfaction, and system performance metrics demonstrate that the application is responsive and reliable for practical use. Throughout the project, we learned several important lessons. Comprehensive testing with real-world diagrams early in development would have revealed platform limitations and optimization opportunities sooner, potentially saving significant refactoring efforts. Our flexibility in technology choices, including the transition from Claude to Gemini, proved valuable in delivering a higher-quality final product and highlighted the importance of data-driven decision-making, even when it requires changing course. Informal feedback from early users provided unanticipated insights and reinforced the value of involving end users throughout the development process. Meanwhile, our iterative approach of building, testing, and refining allowed us to identify and address issues progressively rather than facing a large-scale debugging effort at the end. Maintaining clear documentation and organized code facilitated collaboration and will support future maintenance, making the project more accessible and sustainable. If we were to start the project again, we would conduct a broader comparison of AI models using real OPM diagrams, define clear evaluation criteria and test cases before selecting a platform, implement a comprehensive automated testing suite, and incorporate caching and performance optimization into the initial architecture. Despite areas for improvement, we are proud of the outcome: the OPM Code Generator successfully demonstrates the feasibility and value of AI-powered automated translation from system models to executable code and serves as a practical tool that can significantly accelerate software development workflows, while

also acting as an educational bridge between modeling theory and practical implementation.

7. Project Metrics

At the outset of the project, we established clear success criteria to evaluate the effectiveness and reliability of the OPM Code Generator system. While we defined multiple metrics for comprehensive evaluation, we focused our testing efforts on two critical aspects: user experience and system performance.

7.1 User Experience and Satisfaction

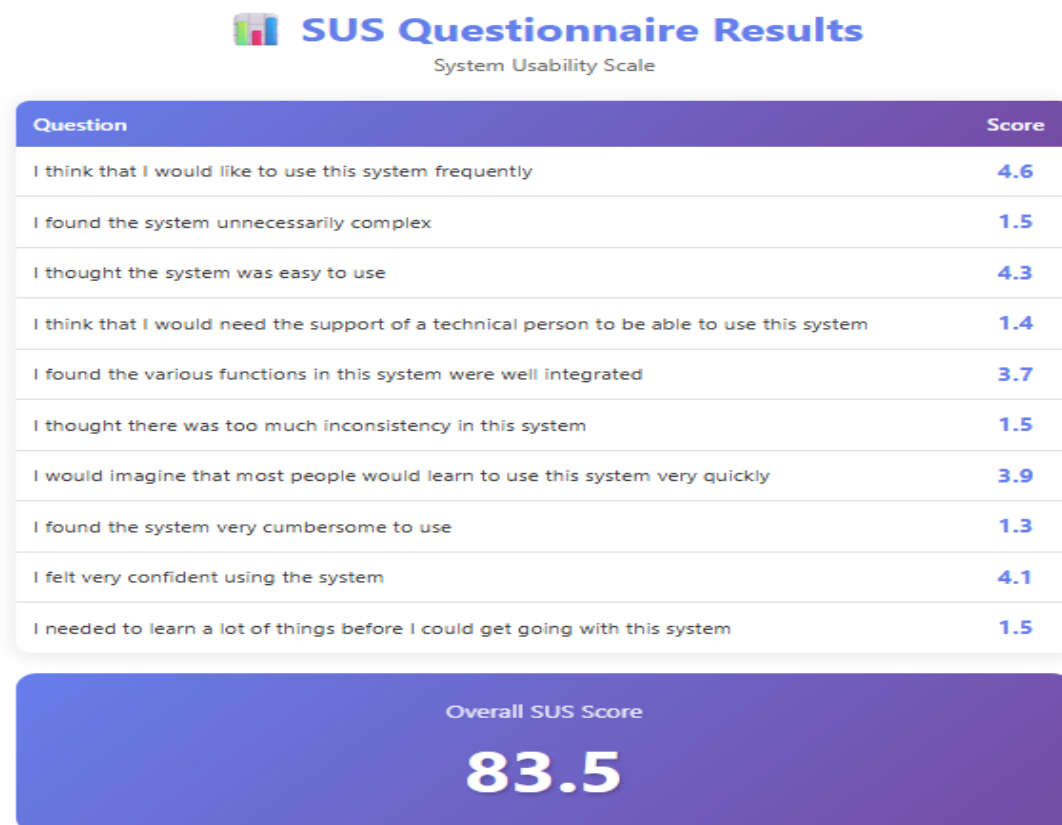


Figure 4: SUS Results

To assess user satisfaction, we conducted a System Usability Scale (SUS) evaluation with a group of 10 participants including software engineering students and developers familiar with OPM methodology. The SUS is a widely recognized standardized questionnaire that measures perceived usability

through ten questions scored on a five-point scale. Our system achieved an SUS score of 83.5, which falls within the "Excellent" range (scores above 80.3 are considered excellent according to industry standards). This score indicates strong user satisfaction with the system's interface, ease of use, and overall functionality. Participants particularly appreciated the intuitive drag-and-drop file upload mechanism, clear visual feedback through loading states and notifications, and the straightforward workflow from diagram upload to code generation. The high SUS score validates our design decisions and confirms that the system successfully delivers a user-friendly experience that meets our objective of creating an accessible tool for translating OPM models into executable code.

7.2 System Performance and Response Time

System performance and reliability were identified as critical metrics for ensuring the system's practical usability in real-world development scenarios. Throughout our development and testing phases, the system demonstrated excellent reliability with minimal downtime, primarily limited to planned maintenance and updates. The FastAPI backend's asynchronous capabilities and MongoDB's scalable architecture have proven capable of handling concurrent requests efficiently. We measured end-to-end processing time from diagram upload to code generation completion across various diagram complexities and programming languages. Response times typically range from 30-60 seconds depending on diagram complexity and selected programming language, with simpler diagrams processing in as little as 20-25 seconds and more complex diagrams occasionally extending to 70-80 seconds. While these times represent a slight delay compared to instant feedback, they align well with user expectations for AI-powered generation tasks, especially considering that manual translation of an OPM diagram to code would take hours or even days. Users receive real-time feedback during processing through animated loading indicators, ensuring they remain informed about system status. The system gracefully handles errors through comprehensive exception management, providing clear feedback to users when issues arise and maintaining system stability even when processing malformed or highly complex inputs.

Overall, based on these two key metrics, the OPM Code Generator has successfully demonstrated both strong user satisfaction and acceptable system performance, validating the core value proposition of automated OPM-to-code translation as a practical tool for software engineering teams.

7.3 Future Work

While the OPM Code Generator successfully demonstrates the feasibility and value of automated OPM-to-code translation, several opportunities exist for future enhancement and expansion. Expanding programming language support to include TypeScript, Go, and Rust would broaden the system's applicability. Enhancing code generation to produce more complete implementations with basic logic and design patterns would provide developers with stronger starting points. Integrating support for Object-Process Language (OPL), the textual representation of OPM, would allow users to input specifications in text format alongside visual diagrams, offering greater flexibility. Adding collaborative features for team-based projects and conducting extensive user studies with professional development teams would further validate the system's practical value and guide future improvements.

7.4 Conclusion

The OPM Code Generator successfully achieved its primary objective of creating an AI-powered system that automates the translation of OPM diagrams into executable code skeletons across multiple programming languages. Through systematic development and iterative refinement, we delivered a functional web application that bridges the gap between high-level system modeling and practical software implementation. The system's architecture, built on React, FastAPI, MongoDB, and Google Gemini AI, demonstrates scalability, reliability, and strong performance. Our evaluation results validate the system's effectiveness, with a System Usability Scale score of 83.5 confirming an intuitive user experience and response times of 30-60 seconds proving acceptable for practical deployment. The project demonstrates that AI-powered automation can effectively translate between abstract system models and concrete code implementations, serving students, architects, development teams, and educators alike. While opportunities for enhancement remain, the current system stands as a complete and valuable tool ready for real-world deployment, contributing to more efficient and collaborative software engineering practices.

8. References

- [1] Levi-Soskin, N., Jbara, A., & Dori, D. (2020). The model fidelity hierarchy: From text to conceptual, computational, and executable model. *IEEE Systems Journal*, 15(1), 1287-1298.
- [2] Li, L., Soskin, N. L., Jbara, A., Karpel, M., & Dori, D. (2019). Model-based systems engineering for aircraft design with dynamic landing constraints using object-process methodology. *IEEE Access*, 7, 61494-61511.
- [3] Forsberg, K., & Mooz, H. (1991). The relationship of system engineering to the project cycle. *Center for Systems Management*, 5333.
- [4] Sulistyo, S., & Najib, W. (2002). Executable uml. *Dept of Information and Communications Technology Agder University College, Norway*.
- [5] Luz, M. P., & Da Silva, A. R. (2004, October). Executing UML Models. In *3rd Workshop in Software Model Engineering (WiSME 2004)* (Vol. 192).
- [6] Firesmith, D. (2003). Modern requirements specification. *Journal of Object Technology*, 2(2), 53-64.
- [7] Levi-Soskin, N., Marwedel, S., Jbara, A., & Dori, D. (2024). Enhancing conceptual models with computational capabilities: A methodical approach to executable integrative modeling. *Systems Engineering*, 27(4), 759-770.

Appendices

Appendix A – User Guide

<u>A.1 Introduction</u>	23
<u>A.2 Operating Instructions</u>	23
<u>A.2.1 Sign-Up Page</u>	25
<u>A.2.2 Login Page</u>	26
<u>A.2.3 OPM Code Generator Page</u>	27
<u>A.2.4 Loading Page</u>	28
<u>A.2.5 OPM Success Page</u>	29
<u>A.2.6 Fix Instructions Page</u>	30
<u>A.2.7 User Projects Page</u>	31

<u>A.2.8 Project View Page</u>	33
<u>A.2.9 About Page</u>	34
<u>A.3 Navigation and Settings</u>	35
<u>A.3.1 Navigation Bar</u>	35
<u>A.3.2 Dark Mode</u>	35
<u>A.3.3 Logging Out</u>	35

A.1 Introduction

The OPM Code Generator is an innovative web-based system designed to bridge the gap between system design and software implementation. This tool automatically transforms Object-Process Methodology (OPM) diagrams into executable code skeletons across multiple programming languages, including Python, Java, C#, and C++. OPM is a widely used methodology for system specification that provides a visual and textual representation of system structure and behavior. By automating the translation from OPM diagrams to code, this system eliminates the tedious manual translation process, allowing software engineers, system architects, and development teams to move from design to development faster and with fewer errors. This user guide provides step-by-step instructions for using the OPM Code Generator system. It covers the complete workflow from creating an account to downloading your generated code. The guide focuses on the nominal (successful) usage path and demonstrates how AI streamlines modern software development workflows.

A.2 Operating Instructions

Today, OpmCodeGenerator runs on two deployment platforms.

The frontend runs on Vercel, you can find it here:

https://opm-code-generator.vercel.app/opm_code_generator

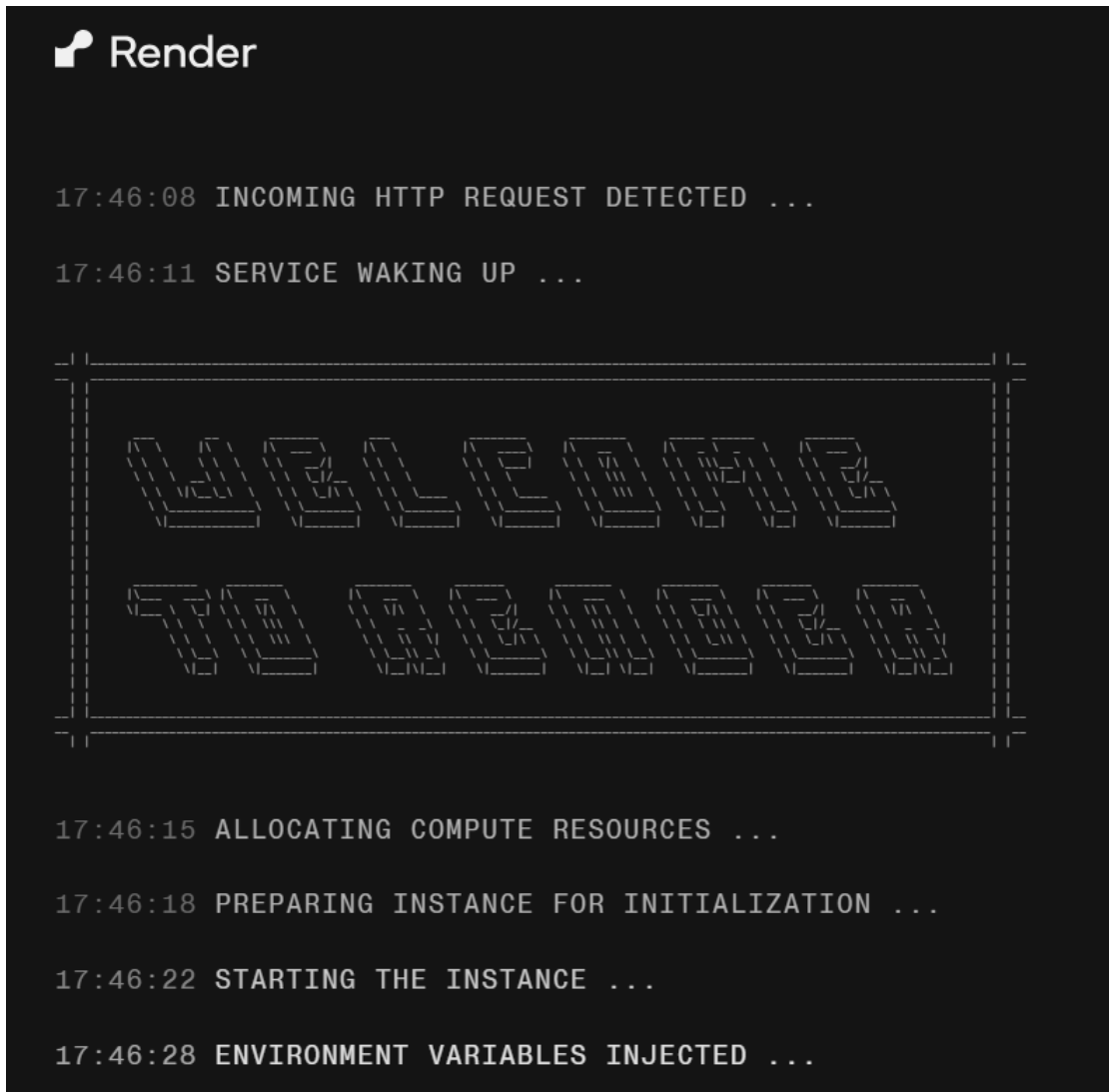
The backend runs on Render, you can find it here:

<https://opm-code-generator-backend.onrender.com/>

Because we are not paying for the deployment, if Render does not recognize a ping to the server more than 15 minutes it turns out the server.

Next time you request something from the server, Render will turn it on and it may take some time.

You will see this:

A screenshot of a terminal window with a dark background and light gray text. At the top left, there is a 'Render' logo and the word 'Render'. The log shows a sequence of events: '17:46:08 INCOMING HTTP REQUEST DETECTED ...', '17:46:11 SERVICE WAKING UP ...', a large 'WELCOME' message in a pixelated font inside a dashed border, '17:46:15 ALLOCATING COMPUTE RESOURCES ...', '17:46:18 PREPARING INSTANCE FOR INITIALIZATION ...', '17:46:22 STARTING THE INSTANCE ...', and '17:46:28 ENVIRONMENT VARIABLES INJECTED ...'.

```
Render

17:46:08 INCOMING HTTP REQUEST DETECTED ...

17:46:11 SERVICE WAKING UP ...

WELCOME

17:46:15 ALLOCATING COMPUTE RESOURCES ...

17:46:18 PREPARING INSTANCE FOR INITIALIZATION ...

17:46:22 STARTING THE INSTANCE ...

17:46:28 ENVIRONMENT VARIABLES INJECTED ...
```

It tells you the backend has been asleep and now Render turns it on.

If you see the next message when you enter the backend url:

```
{"message": "OPM Code Generator API"}
```

It means the server is alive and you can request it and take responses in real time.

The frontend is always alive.

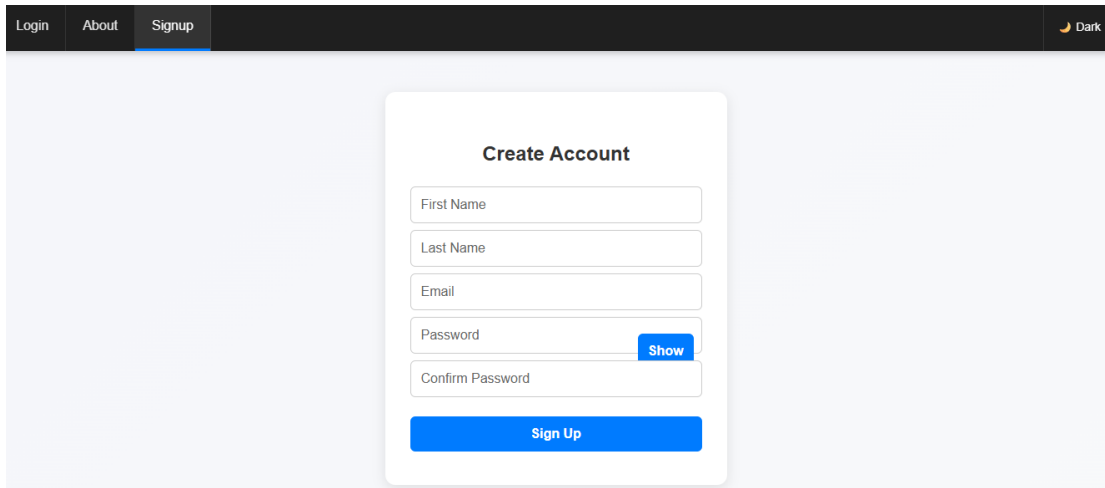
Gemini AI Usage Notes:

This project uses the free tier of the Gemini AI API, which comes with:

- A limited number of tokens per day.
- Restrictions on prompt size and frequency.

If you exceed these limits, API requests may fail or be delayed until the quota resets.

A.2.1 Sign-Up Page



The screenshot shows the 'Sign-Up' page of the BRAUDE College of Engineering, Karmiel. The page has a dark navigation bar at the top with links for 'Login', 'About', and 'Signup'. The 'Signup' link is highlighted. A 'Dark' mode toggle is visible in the top right corner. The main content area is light blue and features a white 'Create Account' form. The form includes input fields for 'First Name', 'Last Name', 'Email', 'Password', and 'Confirm Password'. A 'Show' button is located next to the 'Password' field. A large blue 'Sign Up' button is at the bottom of the form.

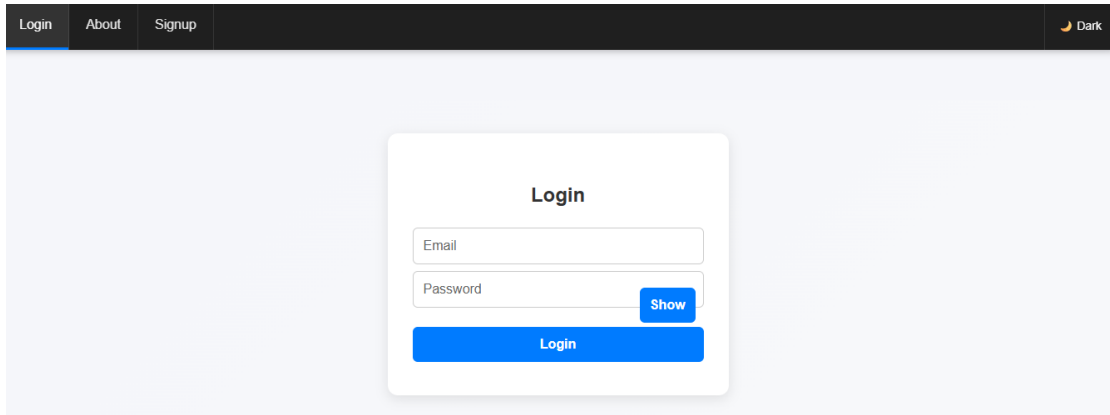
Figure 5: Sign-Up Page

To use the OPM Code Generator, you must first create an account.

Steps to Sign Up:

1. Click on the **Signup** option in the navigation bar
2. You will be directed to the Signup page
3. Fill in the registration form with the following information:
 - **First Name:** Enter your first name
 - **Last Name:** Enter your last name
 - **Email:** Provide a valid email address (this will serve as your username)
 - **Password:** Create a secure password
 - **Confirm Password:** Re-enter your password to confirm
4. Use the **Show** button next to the password fields to toggle password visibility if needed
5. Click the **Sign Up** button to complete the registration

A.2.2 Login Page



The screenshot shows the login page of the BRAUDE system. At the top, there is a dark navigation bar with links for 'Login', 'About', and 'Signup'. The 'Login' link is highlighted. On the right side of the navigation bar, there is a 'Dark' mode toggle. The main content area is light gray and contains a white login form. The form has a title 'Login' at the top. Below the title, there are two input fields: 'Email' and 'Password'. The 'Password' field has a 'Show' button next to it. At the bottom of the form, there is a blue 'Login' button.

Figure 6: Login Page

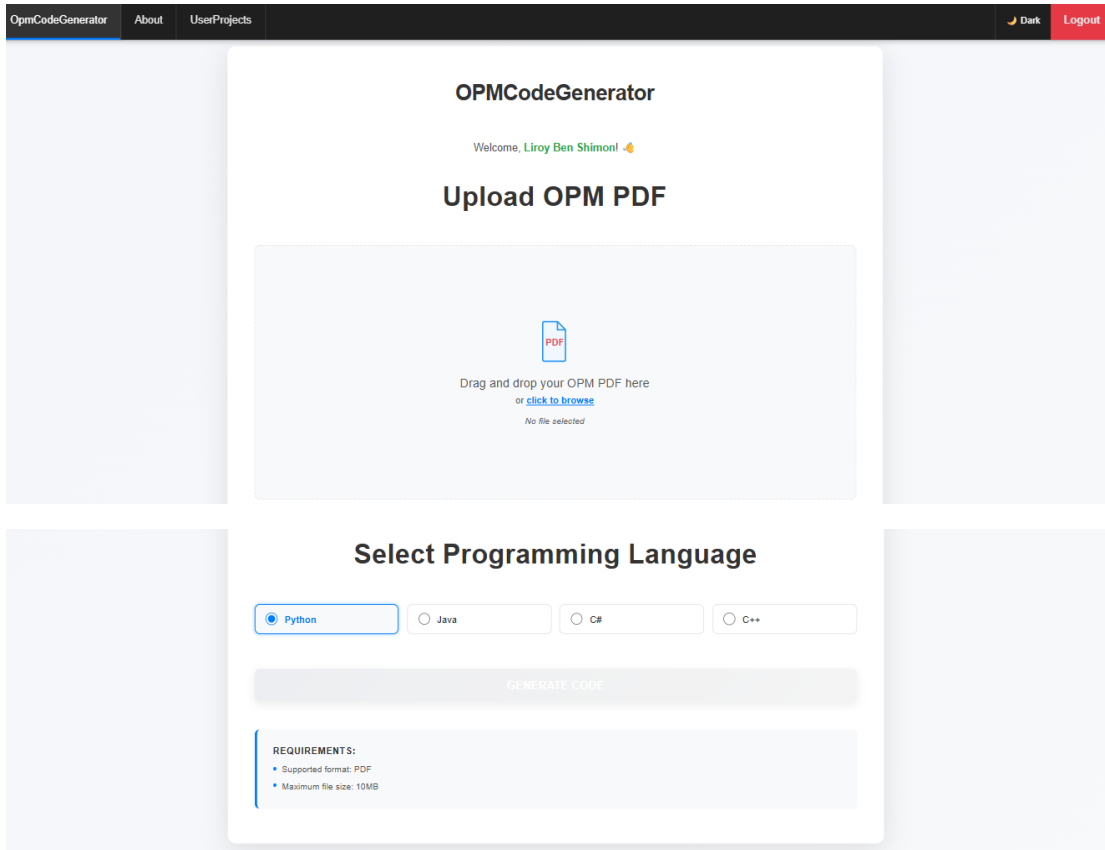
After creating your account, you can access the system by logging in with your credentials.

Steps to Log In:

1. Click **Login** in the navigation bar
2. You will be directed to the Login page
3. Enter your registered **Email** address
4. Enter your **Password** (you can click **Show** to view the password as you type)
5. Click the **Login** button

Upon successful authentication, you will be redirected to the main application page where you can begin generating code from your OPM diagrams.

A.2.3 OPM Code Generator Page



The screenshot shows the OPMCodeGenerator web application. At the top, there is a navigation bar with links for 'OpmCodeGenerator', 'About', and 'UserProjects'. On the right side of the navigation bar, there are links for 'Dark' and 'Logout'. The main content area is divided into two sections. The top section, titled 'OPMCodeGenerator', displays a personalized welcome message: 'Welcome, Liroy Ben Shimon!'. Below this, there is a section titled 'Upload OPM PDF'. It features a large light gray box with a PDF icon and the text 'Drag and drop your OPM PDF here or [click to browse](#)'. Below the box, it says 'No file selected'. The bottom section, titled 'Select Programming Language', contains four radio button options: 'Python' (selected), 'Java', 'C#', and 'C++'. Below these options is a 'GENERATE CODE' button. At the bottom, there is a 'REQUIREMENTS' section with two bullet points: 'Supported format: PDF' and 'Maximum file size: 10MB'.

Figure 7: OPM Code Generator Page

After logging in, you will see a personalized welcome message displaying your name.

The main interface provides an intuitive file upload area for your OPM diagrams.

1. Upload Options:

- **Drag and Drop:** Drag your OPM PDF file directly into the upload area
- **Browse:** Click on "**click to browse**" to select the file from your computer using a file browser

File Requirements:

- **Supported format:** PDF
- **Maximum file size:** 10MB

The system displays "**No file selected**" until you upload a valid OPM PDF document.

2. Select Programming Language:

Before generating code, you must select your target programming language. The system supports four major programming languages: Python (selected by default), Java, C# and C++. Click on the radio button next to your preferred language.

2. Click the **GENERATE CODE** button

A.2.4 Loading Page

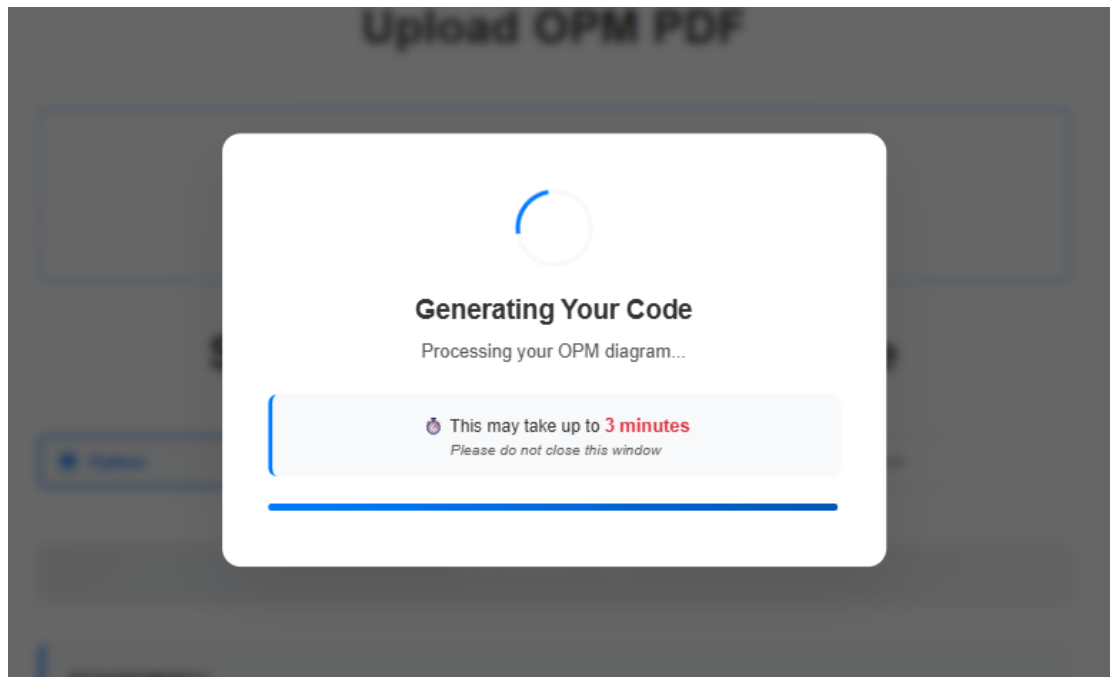


Figure 8: Loading Page

Once you've uploaded your OPM diagram and selected a programming language, the system will process your request and generate the corresponding code.

During the generation process, you will see a modal dialog displaying:

- A loading animation (spinning circular progress indicator)
- The message "Generating Your Code"
- Status Text "Processing your OPM diagram..."
- A notice that "This may take up to 3 minutes" (displayed in red)
- A Warning "Please do not close this window"
- A progress bar showing the current generation status

Important: Do not close the browser window or navigate away during code generation. The AI-powered processing typically completes within 1-3 minutes, depending on the complexity of your OPM diagram.

A.2.5 OPM Success Page

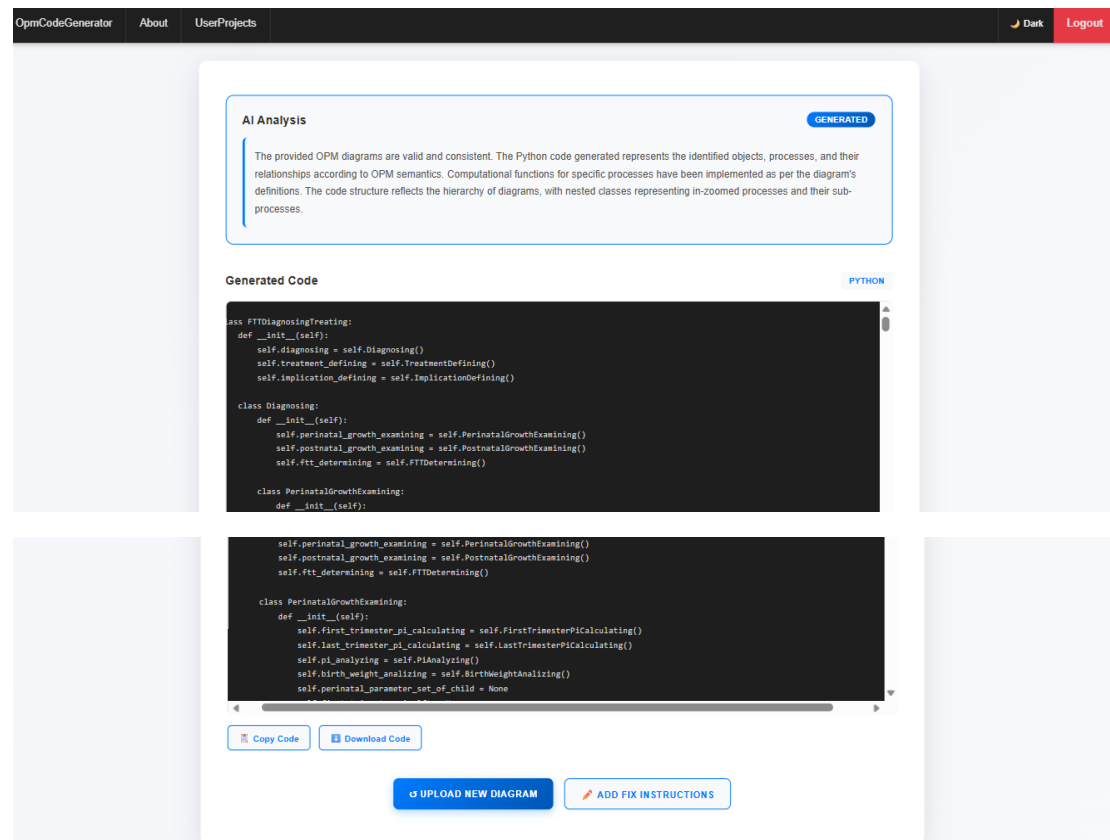


Figure 9: OPM Success Page

1. AI Analysis and Code Display

Once code generation is complete, the system displays two main sections:

- **AI Analysis:** The top section displays an AI-powered analysis of your OPM diagram with a blue "**GENERATED**" badge.
- **Generated Code:** Below the AI analysis, the generated code is displayed in a syntax-highlighted code viewer.

Features:

- Header indicating the programming language (e.g., "PYTHON")
- Dark theme code viewer with syntax highlighting

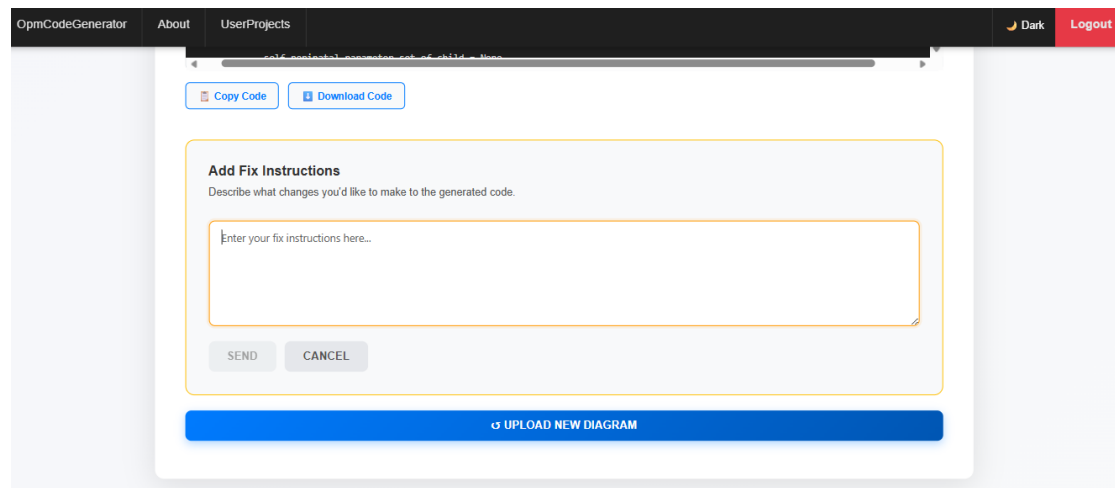
- Scrollable view for examining the complete code structure
- Class definitions, methods, and code organization based on your OPM diagram structure

2.Code Actions

Below the generated code display, you have several action buttons:

- **Copy Code:** Copies the entire generated code to your clipboard for easy pasting into your IDE or text editor
- **Download Code:** Downloads the generated code as a file with the appropriate extension (py, .java, .cs, .cpp)
- **UPLOAD NEW DIAGRAM:** Returns to the upload page to generate code for a different OPM diagram
- **ADD FIX INSTRUCTIONS:** Opens a form to provide feedback or request modifications to the generated code

A.2.6 Fix Instructions Page



The screenshot shows a web application interface for 'OpmCodeGenerator'. At the top, there is a navigation bar with links for 'OpmCodeGenerator', 'About', and 'UserProjects', along with a 'Dark' theme toggle and a 'Logout' button. Below the navigation bar, there are two buttons: 'Copy Code' and 'Download Code'. The main focus is a modal window titled 'Add Fix Instructions'. This modal contains a text area with the placeholder 'Enter your fix instructions here...' and a description below it: 'Describe what changes you'd like to make to the generated code.' At the bottom of the modal, there are 'SEND' and 'CANCEL' buttons. Below the modal, there is a prominent blue button labeled 'UPLOAD NEW DIAGRAM'.

Figure 10: Fix Instructions Page

When you click **ADD FIX INSTRUCTIONS**, a form appears with the following features:

- **Title:** "Add Fix Instructions"
- **Description:** "Describe what changes you'd like to make to the generated code"

- **Text Area:** Large input field with placeholder "Enter your fix instructions here..."
- **SEND** button: Submit your instructions
- **CANCEL** button: Close the form without submitting

You can describe specific changes, improvements, or corrections you would like the AI to apply to the generated code. After submitting, the system will regenerate the code incorporating your feedback.

Below the fix instructions form, the **UPLOAD NEW DIAGRAM** button remains available.

A.2.7 User Projects Page

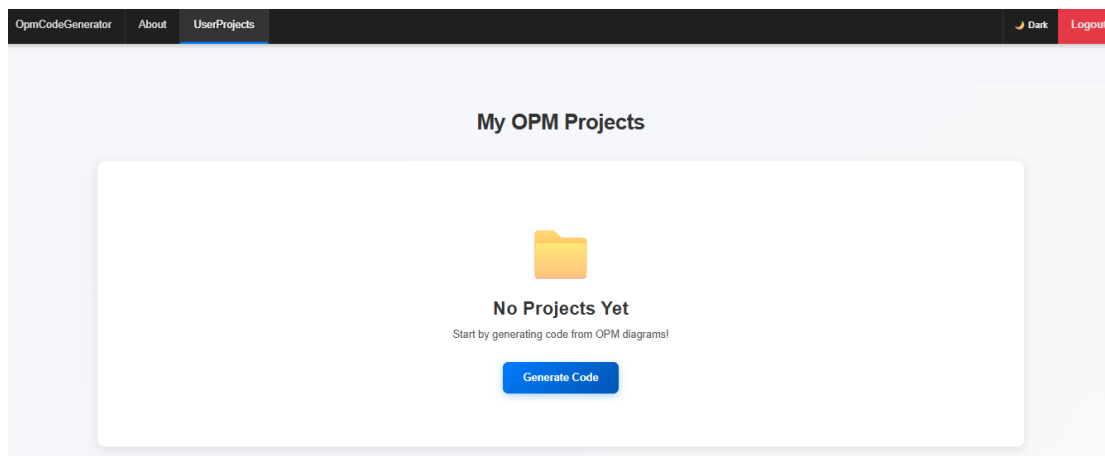


Figure 11: User Projects Page – No Projects Yet

The OPM Code Generator automatically saves all your generated code as projects. Click on **UserProjects** in the navigation bar to view all your saved projects. If you have not yet created a project, the screen will be blank.

The screen includes:

- The message "Start by generating code from OPM diagram!"
- **Generate Code** button: clicking it leads directly to the OPM Code Generator page.

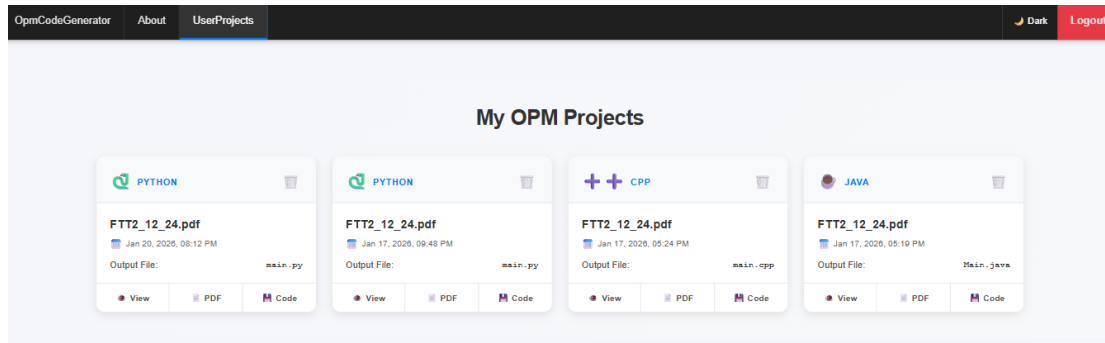


Figure 12: User Projects Page – Existing Projects

If you have already created projects, you can see them on this screen.

Project Cards Display:

Each project is shown as a card containing:

- **Language Badge:** Icon and name of the programming language (Python, Java, C++, or C#)
- **Delete Icon:** Trash can icon in the top-right corner
- **Project Name:** Filename of the original OPM PDF (e.g., "FTT2_12_24.pdf")
- **Creation Date:** When the project was created (e.g., "Jan 20, 2026, 08:12 PM")
- **Output File:** Generated code filename with extension (e.g., "main.py", "Main.java", "main.cpp")
- **Action Buttons:**
 - **View:** Opens detailed project view
 - **PDF:** Download the original OPM diagram PDF file
 - **Code:** Download the generated code file with appropriate extension

The projects are displayed in a grid layout, allowing you to easily browse through all your generated code projects.

A.2.8 Project View Page

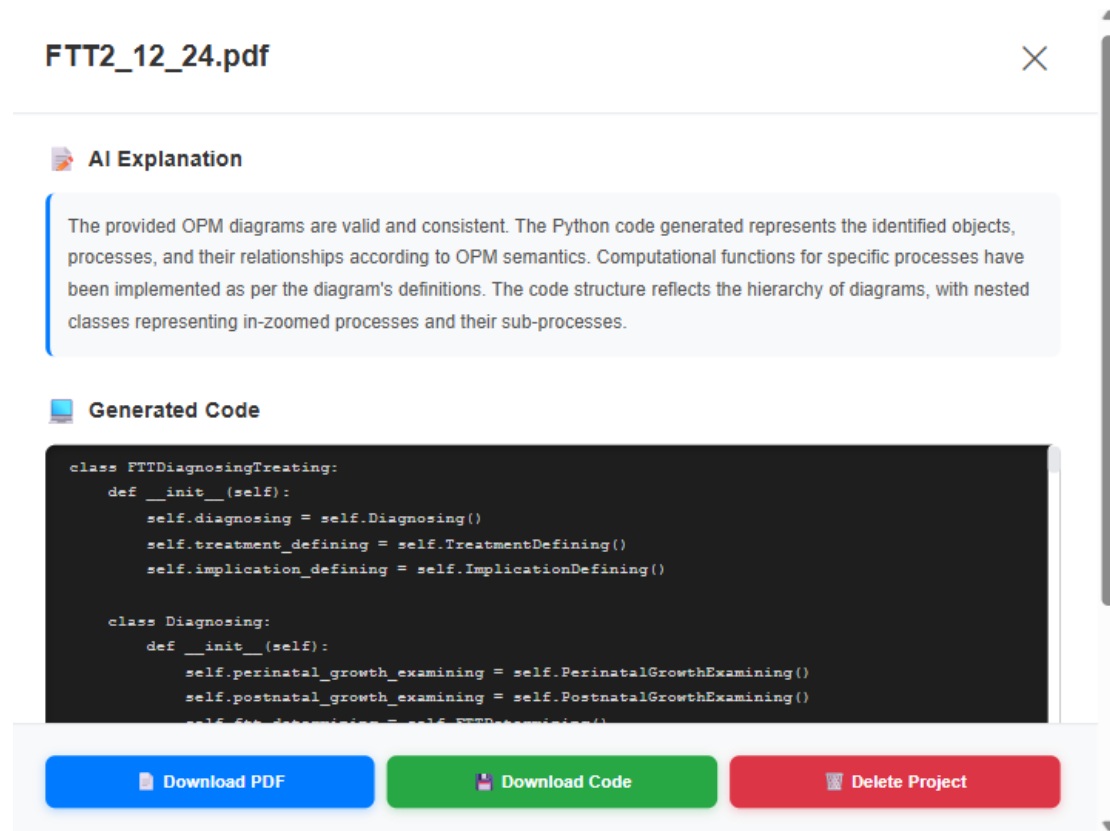


Figure 13: Project View Page

Click the **View** button on any project card to open a detailed modal view.

Project Details Modal Structure:

Header

- Project filename as the title
- Close button (X) in the top-right corner

AI Explanation Section

- Complete AI analysis of the OPM diagram
- Explanation of code structure and implementation

Generated Code Section

- Complete generated code displayed with syntax highlighting
- Dark-themed code viewer
- Scrollable interface

Action Buttons (Bottom of Modal)

Three prominent colored buttons:

1. **Download PDF:** Download the original OPM diagram PDF file
2. **Download Code:** Download the generated code file with appropriate extension
3. **Delete Project:** Permanently remove the project from your account

A.2.9 About Page

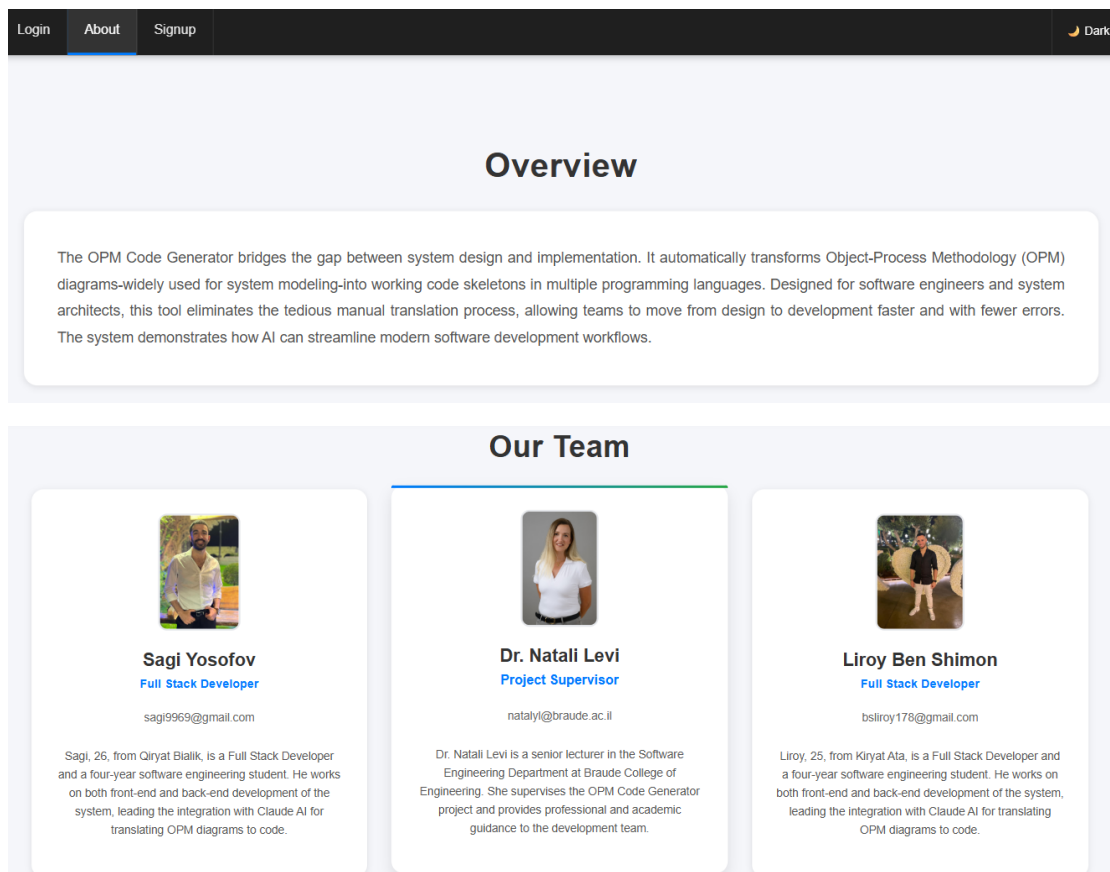


Figure 14: About Page

When you first access the OPM Code Generator, you will land on the About page. This page provides an overview explaining the system's purpose.

Additionally, the page shows the project team members. Each team member is shown with their photo, role, and a brief description of their contribution to the project.

For additional support or questions about the system, please refer to the team contact information available on the About page or contact the project supervisor, Dr. Natali Levi.

A.3 Navigation and Settings

A.3.1 Navigation Bar

The navigation bar remains consistent across all pages (after login) and provides quick access to key areas:

- **OpmCodeGenerator:** Returns to the main upload page
- **About:** View system information and team details
- **UserProjects:** Access your saved projects
- **Dark Mode Toggle:** Switch between light and dark themes
- **Logout:** Sign out of your account

A.3.2 Dark Mode

The system includes a dark mode feature accessible from any page via the moon/sun icon in the top-right corner of the navigation bar.

Benefits of Dark Mode:

- Reduced eye strain during extended use
- Better visibility in low-light environments
- Professional dark theme for code viewing
- Consistent themes across all pages and components

Your theme preference is saved and persists across sessions.

A.3.3 Logging Out

When you're finished using the system, click the **Logout** button in the top-right corner of the navigation bar.

Logout Actions:

- Your session securely ends
- Returns you to the Login page

- Protects your account and projects from unauthorized access

All your projects are saved to your account and will be available the next time you log in.

Appendix B – Maintenance Guide

<u>B.1 Introduction</u>	36
<u>B.2 System Overview</u>	37
<u>B.3 Technologies Used</u>	37
<u>B.4 System Structure</u>	38
<u>B.5 Installation and Setup</u>	39
<u>B.5.1 Prerequisites</u>	39
<u>B.5.2 Local Setup</u>	39
<u>B.5.2.1 Import the Project</u>	39
<u>B.5.2.2 Install Dependencies</u>	39
<u>B.5.2.3 Configuration of Environment Variables</u>	40
<u>B.5.2.4 Run the Project</u>	40
<u>B.6 Deployment</u>	41
<u>B.6.1 Redeployment</u>	43
<u>B.6.2 Environment Variables</u>	43
<u>B.7 Common Maintenance Tasks</u>	43
<u>B.7.1 Updating Dependencies</u>	43
<u>B.7.2 GitHub</u>	44
<u>B.8 Conclusion</u>	44

B.1 Introduction

This maintenance guide is intended to help future developers, maintainers, or instructors understand how to maintain, update, and extend the OPM Code Generator project.

The document explains the system structure, main components, common maintenance tasks, and recommends best practices. It is written to support long-term maintainability of the system.

B.2 System Overview

OPM Code Generator is a web-based application that uses AI to automatically generate source code from OPM (Object-Process Methodology) diagrams.

The system allows users to:

- Upload or select OPM diagrams
- Process and analyze diagram structure
- Generate corresponding source code automatically
- Manage projects and generated outputs

The system follows a client–server architecture and integrates AI-based logic for code generation.

B.3 Technologies Used

Frontend

- React
- Vite (development and build tool)
- JavaScript
- HTML and CSS (used within React components)

Backend

- Python (FastApi)

AI

- Gemini Agent (via API key)

Database

- MongoDB

Version Control

- GitHub

Deployment

- Vercel (frontend deployment)

- Render (backend deployment)

B.4 System Structure

```

opm-code-generator/
├── backend/
│   ├── ai/ # AI-related logic and agents
│   ├── assets/ # contains the OPM files for teaching the ai agent.
│   ├── db/ # Database connection and configuration
│   ├── models/ # Data models and schemas
│   ├── routers/ # FastAPI route definitions
│   ├── __init__.py # Backend package initialization
│   ├── definitions.py # System-wide constants and definitions
│   ├── main.py # FastAPI application entry point
│   ├── requirements.txt # Python dependencies
│   └── .gitignore # Backend Git ignore rules
├── frontend/
│   ├── public/ # Public static assets
│   ├── src/
│   │   ├── api/ # API communication layer
│   │   ├── components/ # Reusable React components
│   │   ├── context/ # React context and global state
│   │   ├── pages/ # Application pages
│   │   ├── styles/ # CSS and styling files
│   │   ├── App.css # Global application styles
│   │   ├── App.jsx # Root React component
│   │   ├── index.css # Base CSS styles
│   │   └── main.jsx # React application entry point
│   ├── eslint.config.js # ESLint configuration
│   ├── index.html # HTML entry file
│   ├── package.json # Frontend dependencies and scripts
│   ├── package-lock.json # Dependency lock file
│   ├── vercel.json # Vercel deployment configuration
│   ├── vite.config.js # Vite configuration
│   └── .gitignore # Frontend Git ignore rules
└── README.md # Project overview and basic instructions
  
```

B.5 Installation and Setup

B.5.1 Prerequisites

Before running the system, ensure the following software is installed:

python 3.12 or higher – Required for the FastAPI backend and asynchronous AI agent handling.

Node.js v20 or higher – Required for the React + Vite frontend development environment.

Git – Used for version control and deployment (e.g., Vercel, Render).

In addition, ensure you have internet connection for accessing the Gemini AI agent.

B.5.2 Local Setup

B.5.2.1 Import the Project

Clone the repository: `git clone https://github.com/SagiYosofov/opm-code-generator.git`

After cloning the repository, the project is accessed locally via localhost URLs during development, and via the deployed URLs in production (Vercel for frontend, Render for backend).

Navigate to the root directory, run `"cd opm-code-generator"`

B.5.2.2 Install Dependencies

For frontend:

- Navigate to the frontend directory from the root: Run `"cd frontend"`
- Install dependencies: Run `"npm install"`

For backend:

- Navigate to the backend directory from the root: Run `"cd backend"`
- Create a virtual environment: Run `"python -m venv .venv"`
- Activate the virtual environment:

- On Windows: Run ".venv\Scripts\activate"
- On macOS/Linux: Run "source .venv/bin/activate"
- Install dependencies: Run "pip install -r requirements.txt"

B.5.2.3 Configuration of Environment Variables

The system uses environment variables to manage configuration values and sensitive information. These variables are stored in .env files and must not be committed to version control.

Frontend Environment Variables

Located in frontend/.env:

- VITE_API_URL – Base URL of the backend API used by the frontend for all server communication.

Backend Environment Variables

Located in backend/.env:

- FRONTEND_URL = URL of the frontend application.
- MONGO_URI – MongoDB connection string.
- MONGO_DB_NAME – Name of the MongoDB database used by the application.
- GEMINI_API_KEY – API key for accessing the Gemini AI service.

Proper configuration of these variables is required for the system to function correctly in both development and production environments.

B.5.2.4 Run the Project

Follow the steps below to run the system locally in a development environment.

Backend Side

Open the backend folder in your IDE (don't use "cd backend" from the root directory but open it in a separate window).

Start the FastAPI server: Run "uvicorn main:app --reload"

The backend API will be available by default at <http://localhost:8000>.

Example:

```
(.venv) PS C:\Users\sagi9\Desktop\ב\שלב ב\פרויקט סוף - שנה ד\מסמך\שנה ד\הנדסת תכנה\שנה ד\מסמך\שנה ד\פרויקט סוף - שנה ב\opm-code-generator\backend> uvicorn main:app
INFO: Will watch for changes in these directories: ['C:\Users\sagi9\Desktop\ב\שלב ב\פרויקט סוף - שנה ד\מסמך\שנה ד\הנדסת תכנה\שנה ד\מסמך\שנה ד\פרויקט סוף - שנה ב\opm-code-generator\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [32036] using StatReload
C:\Users\sagi9\Desktop\ב\שלב ב\פרויקט סוף - שנה ד\מסמך\שנה ד\הנדסת תכנה\שנה ד\מסמך\שנה ד\פרויקט סוף - שנה ב\opm-code-generator\backend\.venv\Lib\site-packages\pymon
:352: CryptographyDeprecationWarning: Parsed a serial number which wasn't positive (i.e., it was negative or zero), which is
. Loading this certificate will cause an exception in a future release of cryptography.
crypto.X509.from_cryptography(x509.Load_der_x509_certificate(cert))
INFO: Started server process [26576]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Frontend Side

In a different window, open the project in the root directory "opm-code-generator".

Navigate to the frontend folder: Run "cd frontend"

Start the development server: Run "npm run dev"

The frontend application will be available at the local URL provided by Vite (typically <http://localhost:5173>).

Example:

```
PS C:\Users\sagi9\Desktop\ב\שלב ב\פרויקט סוף - שנה ד\מסמך\שנה ד\הנדסת תכנה\שנה ד\מסמך\שנה ד\פרויקט סוף - שנה ב\opm-code-generator> cd .\frontend\
PS C:\Users\sagi9\Desktop\ב\שלב ב\פרויקט סוף - שנה ד\מסמך\שנה ד\הנדסת תכנה\שנה ד\מסמך\שנה ד\פרויקט סוף - שנה ב\opm-code-generator\frontend> npm run dev

> frontend@0.0.0 dev
> vite

You are using Node.js 22.11.0. Vite requires Node.js version 20.19+ or 22.12+. Please upgrade your Node.js version.

VITE v7.2.2 ready in 756 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

B.6 Deployment

Today, OpmCodeGenerator runs on two deployment platforms.

The frontend runs on Vercel, you can find it here:

https://opm-code-generator.vercel.app/opm_code_generator

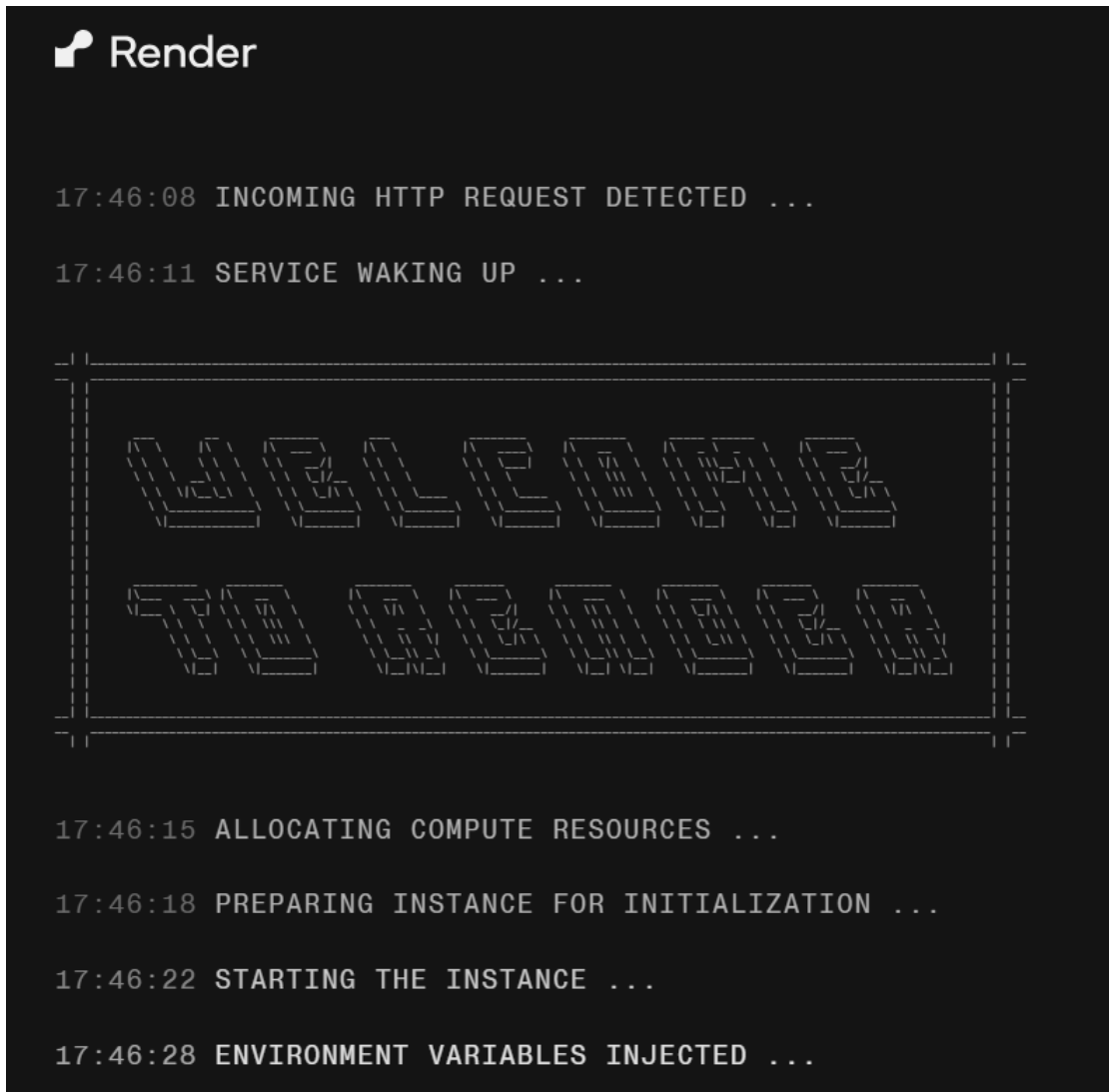
The backend runs on Render, you can find it here:

<https://opm-code-generator-backend.onrender.com/>

Because we are not paying for the deployment, if Render does not recognize a ping to the server more than 15 minutes it turns out the server.

Next time you request something from the server, Render will turn it on and it may take some time.

You will see this:



```
Render

17:46:08 INCOMING HTTP REQUEST DETECTED ...

17:46:11 SERVICE WAKING UP ...

17:46:15 ALLOCATING COMPUTE RESOURCES ...

17:46:18 PREPARING INSTANCE FOR INITIALIZATION ...

17:46:22 STARTING THE INSTANCE ...

17:46:28 ENVIRONMENT VARIABLES INJECTED ...
```

It tells you the backend has been asleep and now Render turns it on.

If you see the next message when you enter the backend url:

```
{"message": "OPM Code Generator API"}
```

It means the server is alive and you can request it and take responses in real time.

The frontend is always alive.

B.6.1 Redeployment

Vercel is connected to the main branch and listens to the frontend folder.

If there are pushes to the main branch and there are changes under the frontend folder, Vercel will rebuild and redeploy automatically in order to add the changes.

Render is connected to the main branch and listens to the backend folder.

If there are pushes to the main branch and there are changes under the backend folder, Render will rebuild and redeploy automatically in order to add the changes.

B.6.2 Environment Variables

We need to enter the deployment platforms to provide the required environment variables.

To Vercel we need to add:

- VITE_API_URL = <https://opm-code-generator-backend.onrender.com>

To Render we need to add:

- FRONTEND_URL – https://opm-code-generator.vercel.app/opm_code_generator
- MONGO_URI – MongoDB connection string.
- MONGO_DB_NAME – Name of the MongoDB database used by the application.
- GEMINI_API_KEY – API key for accessing the Gemini AI service.

B.7 Common Maintenance Tasks

B.7.1 Updating Dependencies

- The frontend dependencies are updated automatically by using the "npm install" command.
You can verify they are really updated by checking the file opm-code-generator\frontend\package.json .

The value of "dependencies" key contains all packages you installed on the frontend side.

- The backend dependencies can be updated by running the command "pip freeze > requirements.txt".

B.7.2 GitHub

GitHub clone url: <https://github.com/SagiYosofov/opm-code-generator.git>

In order to update your local main branch with the new features that were pushed to GitHub, run "git pull origin main". This will update your local main branch with the latest updates pushed to the main branch in GitHub.

If you want to make some changes or add new things, from main branch create new branch. In this branch you will make your updates.

Push your updates to GitHub, run the commands:

```
git add .
```

```
git commit -m "Your commit message"
```

```
git push origin <current branch name>
```

After finishing your updates, create Pull Request and merge the new branch to the main branch.

After merging, Vercel and Render will redeploy automatically if you change the directories they listens to (Vercel listens to the frontend directory and Render listens to the backend directory).

After merging, pull the changes from Github to update your main branch locally and delete the former branch locally and remotely in Github.

B.8 Conclusion

This maintenance guide provides the necessary information to understand, maintain, and extend the OPM Code Generator project. Following the guidelines in this document will help ensure system stability, scalability, and long-term usability.